



useRef in React



useRef is a hook that returns an object with a **.current** property. It doesn't trigger a re-render when its value changes. This makes it different from **useState**, which will re-render the component when updated.

The key uses of **useRef** are:

1. Accessing DOM elements directly.
2. Persisting values across renders without triggering re-renders.

Syntax

```
const myRef = useRef(initialValue);
```

- **initialValue** is the value that you want to initialize the ref with.
- **myRef** is an object that contains the **.current** property, which holds the value.



Key Characteristics of useRef

1. Persistent across renders:

The object returned by `useRef` remains the same between renders. It doesn't change or reset when the component re-renders.

2. Does not cause re-renders:

Updating the `.current` property does not cause the component to re-render, unlike `useState`.

3. Mutable:

You can change the `.current` value anytime, and it doesn't trigger a re-render.



Use Cases

1. Accessing DOM Elements:

One of the most common use cases for `useRef` is to interact with DOM elements directly. This is useful when you need to focus on an input, measure the size of an element, or manage animations.

Example: Focusing an Input Element

```
import React, { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);
  const handleFocus = () => {
    inputRef.current.focus(); // Accesses DOM and focuses input
  };
  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleFocus}>Focus Input</button>
    </div>
  );
}
```



2. Persisting Values Across Renders:

You can store values that should persist across renders, but without triggering a re-render, using `useRef`. This is useful for tracking things like previous values, timeouts, or other non-UI-related data. In below Example we'll Storing Previous State Value

```
import React, { useState, useEffect, useRef } from 'react';

function PreviousState() {
  const [count, setCount] = useState(0);
  const prevCountRef = useRef();
  useEffect(() => {
    prevCountRef.current = count; // Store the previous count
  }, [count]); // Only run when count changes
  return (
    <div>
      <p>Current Count: {count}</p>
      <p>Previous Count: {prevCountRef.current}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```



3. Storing Interval or Timeout IDs:

You can store `setInterval` or `setTimeout` IDs in refs to clear them later without causing re-renders. This is useful for clearing intervals or timeouts when the component is unmounted.

Example: Setting and Clearing Interval

```
import React, { useRef, useEffect } from 'react';

function Timer() {
  const intervalRef = useRef();

  useEffect(() => {
    intervalRef.current = setInterval(() => {
      console.log('Interval running...');
    }, 1000);

    // Cleanup interval when component unmounts
    return () => clearInterval(intervalRef.current);
  }, []); // Run once on mount

  return <div>Timer is running...</div>;
}
```



4. Storing Mutable References for Performance Optimizations:

If you want to store a value that should not trigger a re-render e.g. for instance, a value that will change frequently but doesn't need to affect the component's rendering, `useRef` is perfect for that.

Example: Tracking Mouse Position

```
import React, { useRef, useEffect } from 'react';

function MouseTracker() {
  const mousePositionRef = useRef({ x: 0, y: 0 });

  useEffect(() => {
    const handleMouseMove = (e) => {
      mousePositionRef.current = { x: e.clientX, y: e.clientY };
    };

    window.addEventListener('mousemove', handleMouseMove);
  });
}
```

Code Continue to next Page



```
return () => {  
  window.removeEventListener('mousemove', handleMouseMove);  
};  
}, []);  
  
return (  
  <div>  
    <p>Move the mouse around!</p>  
    <p>Current Mouse Position: `{X: ${mousePositionRef.current.x}, Y:  
${mousePositionRef.current.y}}`</p>  
  </div>  
);  
}
```




5. Tracking Previous Props or Values :

In certain cases, you may need to track previous values of props or component state that are not directly tied to rendering.

Example: Detecting Prop Changes

```
import React, { useRef, useEffect } from 'react';

function PropChangeTracker({ value }) {
  const prevValueRef = useRef();

  useEffect(() => {
    prevValueRef.current = value; // Store the previous value
  }, [value]);

  return (
    <div>
      <p>Current Value: {value}</p>
      <p>Previous Value: {prevValueRef.current}</p>
    </div>
  );
}
```



Key Differences Between useRef and useState

1. Re-renders:

Updating a useRef does not cause a re-render, while updating a useState causes the component to re-render.

2. Value Persistence:

Both useRef and useState persist values across renders, but useRef does so without triggering a re-render.

3. Use Case:

useState is best for values that need to affect the UI, while useRef is best for values that don't directly affect rendering, like DOM nodes or mutable variables.